C# Fundamentals

One-look revision notes | types, records, exceptions, interfaces, LINQ, async, testing & debugging

1 Types | 2 Records | 3 Exceptions | 4 Testable design | 5 Abstract/Iface | 6 Collections
7 LINQ | 8 Async | 9 xUnit | 10 Debugging | 11 Recap

1 Value vs Reference Types & Nullable References

	Value type	Reference type
Examples	`int, bool, double, struct`	`class, string, array`
Holds	The data itself	An address to an object on the heap
Assignment	**Copies data**; variables independent	**Copies the reference**; both point to one object
Passed to method	Changes stay inside the method	Changes persist outside
Can be null?	No	Yes (NullReferenceException if used)

VALUE TYPE: independent copies

```
int x = 5;
int y = x;
y = 10;       // x is still 5
```

REFERENCE TYPE: shared object

```
Person p2 = p1;
p2.Name = "Bob";  // p1.Name is "Bob" too
// need a copy? use new Person{...}
```

- **Nullable Reference Types (C# 8+):** add `#nullable enable`. `string?` may be null; `string` must not be. Moves null bugs to **compile time**.
- Fix **every** warning before deploying. Guard with `if (x != null)`, `x?.Name`, or `x?.Name ?? "Unknown"`.
- `!` (null-forgiving) only when you are **certain** it isn't null; if you're wrong it crashes and the compiler stays silent.

2 Records, Pattern Matching & Switch Expressions

- **Record** = reference type with **value equality** (compares property values, not identity) and built-in ToString/constructor. Immutable by default.
- `record Person(string Name, int Age);` → two records with the same values: `a == b` is **true** (a class gives false). One property differs = not equal.

Pattern	Example	Checks
Type	`o is string s`	Kind of object, binds variable
Property	`p is { Age: > 18, Status: "Active" }`	Values inside an object
Relational	`score is >= 90`	Comparisons
Logical	`n is > 0 and < 100`	`and / or / not`
List	`nums is [1, 2, ..]`	Shape/contents of a collection
when	`p is { Age: var a } when a > 18`	Extra condition

SWITCH EXPRESSION

```
var status = user switch {
    Administrator => "Full access",
    Manager { TeamSize: > 5 } => "Large team lead",
    _ => "Unknown role"          // discard = safety net
};
```

- Always end with _. Keep patterns readable; extract a helper if a teammate would pause.

3 Exceptions: try / catch / finally & When to Throw

Block	Role
`try`	Risky code. Runs until a line throws; the rest of try is skipped
`catch (Type)`	Runs **only if the type matches**. Catch specific types (FormatException, IOException)
`finally`	**Always runs** (even if catch returns/throws): close files, release resources

Choice	Use when	Example
Throw	Unexpected / unrecoverable; broken rule (invariant); caller must decide	DB down, `id <= 0`, UserNotFoundException
Return	Expected, recoverable outcome	`int.TryParse`, login returns false, empty list
Catch	You can actually fix it right here	Bad input → default value

> **Warning:** Never use a bare `catch { }`. It swallows real bugs (e.g. NullReferenceException). Catch what you expect; let the rest bubble up. Throwing fails loudly; returning relies on the caller checking.

4 Classes & Interfaces Designed for Testing

- **Problem:** a class that creates a concrete dependency (`new DatabaseLogger()`) forces slow, side-effect-heavy tests that need real infrastructure.
- **Fix:** depend on an **interface** + **constructor injection** (dependency injection). Production passes the real class; tests pass a **test double** (fake/mock) that is fast, in-memory and side-effect free. The class under test never changes.

```
public interface ILogger { void Log(string message); }

public class Calculator {
    private readonly ILogger _logger;
    public Calculator(ILogger logger) { _logger = logger; }   // injected
}
public class MockLogger : ILogger {                            // test double
    public List<string> Messages { get; } = new();
    public void Log(string m) => Messages.Add(m);
}
```

Sign in code	Testable?
Constructor takes `ILogger`	Yes
Constructor takes a concrete class, or `new ConcreteLogger()` inside	No
Test subclass of the real class	No: parent constructor and dependencies still run

> **Common mistake:** Writing a "test version" of a class by **inheriting** keeps the parent's coupling. Implement the **interface** instead. Prefer composition over inheritance.

5 Abstract Class vs Interface

	Abstract class	Interface
Provides	Shared code, fields, abstract + concrete methods	Contract only (signatures)
Subclass	Must override abstract methods; inherits concrete ones	Must write **every** method itself
Limit	Inherit from **one**; cannot instantiate	Implement **many** at once
Use when	Related types genuinely share code (Character: health logic)	Unrelated types share behaviour (ILogger, IPaymentMethod, INotifier)

- Ask: **do I need to share code, or just define a contract?** Default to an interface; use an abstract class only when shared code saves real effort.
- Interfaces suit third-party plugins and classes that already inherit elsewhere; wrong early choice locks you into one inheritance line.
- You can combine both: abstract base + extra interfaces. Interfaces also make testing easy (swap fakes).

6 List, Dictionary & Generics

Collection	Access by	Use when
`List<T>`	Index (starts at 0), keeps order	Position/order matters, add/remove items
`Dictionary<TKey,TValue>`	Unique key, very fast lookup	Find by ID/email/code (no scanning)
`Queue / Stack / HashSet`	FIFO / LIFO / membership	Arrival order / undo / "does it exist?"

- Choose by **access pattern**, not by what feels easy. A List where you need key lookups scans every item and slows as data grows.
- `list[5]` on a short list crashes: check `Count`. Adding a **duplicate key** throws; update with `dict[key] = value`.
- **Generics** (<T>) = type placeholder: compile-time type safety, no casting, write once and reuse. Return `default(T)` for "nothing" (0, false, null).

```
public T GetFirstOrDefault<T>(List<T> items)
    => items.Count == 0 ? default(T) : items[0];
```

7 LINQ: Where, Select, GroupBy, Any

Operator	Does	Example
`Where`	Filter (predicate)	`p => p.Price > 100`
`Select`	Transform / project	`p => p.Name`
`GroupBy`	Bucket by key; later steps see groups	`o => o.Category`
`Any`	Does at least one match? Stops at first hit	`o => o.Status == "Pending"`
`OrderByDescending`	Sort high to low	`s => s.Grade`

```
var top = students
    .Where(s => s.Grade >= 80)              // filter first
    .Select(s => new { s.Name, s.Grade })   // then reshape
    .OrderByDescending(s => s.Grade)
    .ToList();                              // runs here (deferred execution)
```

- **Lazy:** nothing runs until `ToList()`, `First()`, `Any()` or a loop. A lambda is `parameter => expression`.
- **Order matters:** filter before transform to do less work; filter on a total only *after* `GroupBy`. Plan backwards from the target output.
- Existence check: use `Any()`, **never** `Count() > 0` (scans everything). Any on empty = false.

8 async / await & Async Traps

- **Blocking I/O** on the UI thread freezes the app. `await` **pauses your method, not the thread**; the thread is freed for clicks and redraws, then resumes when I/O completes. Use for web APIs, databases, files.
- **Async method = 3 parts:** `async` keyword, return type `Task` (no value) or `Task<T>` (value), and `await` inside. Name it with the **Async** suffix. Marking `async` without any `await` adds overhead for nothing.

Trap	What goes wrong	Fix
`async void`	Exceptions can't be caught by try/catch; silent failure	`async Task` (void only for event handlers)
`.Result / .Wait()`	Blocks the thread, freezes UI, can deadlock	`await`; keep async all the way up
Fire-and-forget (no await)	Task untracked; failures vanish	`await` it, or `_ = Task` with error logging

BAD

```
var data = GetDataAsync().Result;
async void Load() { await Fetch(); }
```

GOOD

```
var data = await GetDataAsync();
async Task Load() { await Fetch(); }
```

9 Testing with xUnit & Reading Failures

- **Arrange** (set up data/objects) → **Act** (one call to the code under test) → **Assert** (verify). Mark tests with `[Fact]`; check with `Assert.Equal(expected, actual)`.
- Name: `Method_Condition_ExpectedResult` e.g. `Add_WithTwoPositiveNumbers_ReturnsTheirSum`. **One behavior per test**, short Arrange.
- Run with `dotnet test`. Cycle: **run** → **read** → **fix** → **rerun**. Tests also catch regressions.

Read the whole failure, not just the red line

- Output gives: **test name**, **status**, **assertion message**, **Expected vs Actual**, and a **stack trace** (file + line). The gap between Expected and Actual is the clue.

Expected vs Actual	Likely cause
`10` vs `9`	Off-by-one (loop < vs <=)
`42` vs `-42` / `17`	Sign flip / wrong value or operation
`"Success"` vs `"success"`	Case sensitivity
5 items vs 3 items, or `0` vs value	Missing data (filter too strict), unset variable

10 Debugging: Breakpoints & Stepping (VS / VS Code)

- Test output shows **what** failed; the **debugger shows why**. Beats guessing and print statements.
- **Breakpoint:** click the line-number margin (red dot), run Debug (**F5**); code pauses on that line. Place it at the function entry.
- Inspect: **hover** a variable, the **Variables panel**, and the **Call stack** (shows how deep you are). The line where a value first goes wrong is your bug.

Command	Key	Use when
Step Over	F10	You trust the function; run it and move to next line
Step Into	F11	You suspect the bug is inside the call
Step Out	Shift+F11	You've seen enough; finish function, return to caller

Tip: Stepping into library code gets confusing: step out, or set a new breakpoint nearer your own code.

11 Course Recap & Next Steps

Pillar	You can now
Foundation	Types, value vs reference, nullable safety
Structure (OOP)	Classes, records, interfaces vs abstract classes, DI for testability
Responsiveness & data	async/await (no .Result / async void), collections, generics, LINQ
Quality	Exceptions, xUnit tests, reading failures, debugging

Next up: Next: the **five C# challenges** unlock the Week 3 career gate. Treat them as portfolio-building practice.